

CASE STUDY 1

Credential Exposure at the Point of Entry: A Controlled Localhost Phishing Simulation Case Study

By: Joseph Gonzalez



ZERODAY TECH LABS



Scope	Localhost-only simulation using self-generated test credentials and researcher-controlled pages; educational and analytical purposes only
Environment	Kali Linux terminal, locally hosted practice login page, Firefox, SEToolkit, localhost traffic only
Primary Evidence	SET launch screenshot, attack-selection screenshot, source login screenshot, localhost clone screenshot, black-bar redacted terminal capture

For educational and analytical purposes only. Local address details and submitted values shown in figures have been redacted with opaque black bars.

Introduction

Households are attractive targets for credential theft because day-to-day logins happen on personal phones, laptops, tablets, and shared family devices, often under time pressure and with repeated password reuse across services. Traditional awareness advice such as "look carefully at the page" is helpful, but it is not always enough when a cloned interface is visually convincing and the user is already expecting to sign in.

This project uses a case-study driven approach rather than a generic lab checklist. The goal is to examine a single controlled scenario in depth, document what the interface looked like, record what happened when the form was submitted, and explain why the result would matter beyond the lab.

Methodology

A controlled cybersecurity simulation was conducted in an isolated virtual lab environment using Kali Linux, a locally hosted HTML login page, a web browser, and SEToolkit as the instrumentation layer for observing form submission behavior. The scope was limited to localhost traffic and self-generated test data. No real victims, third-party accounts, public hosts, or production systems were involved.

1. Overview

A cloned login page is a deceptive interface that imitates an expected sign-in screen and captures data at the moment the form is submitted. In this case, the analytical question was whether a locally hosted practice page, once cloned and presented back through localhost, would collect self-generated test credentials during a controlled interaction. The case therefore focuses on point-of-entry exposure rather than server-side password storage.

2. Attack Setup

The lab host was used to prepare the controlled credential-capture workflow. As shown in Figure 1, SET was launched in the lab environment. Figure 2 adds the attack-selection view, showing the credential harvester path used for the controlled scenario. Figure 3 shows the source login interface used as the local practice page. The significance of this stage is not the specific account shown; it was that the interface resembled an ordinary login experience closely enough to support normal user interaction during the controlled session.

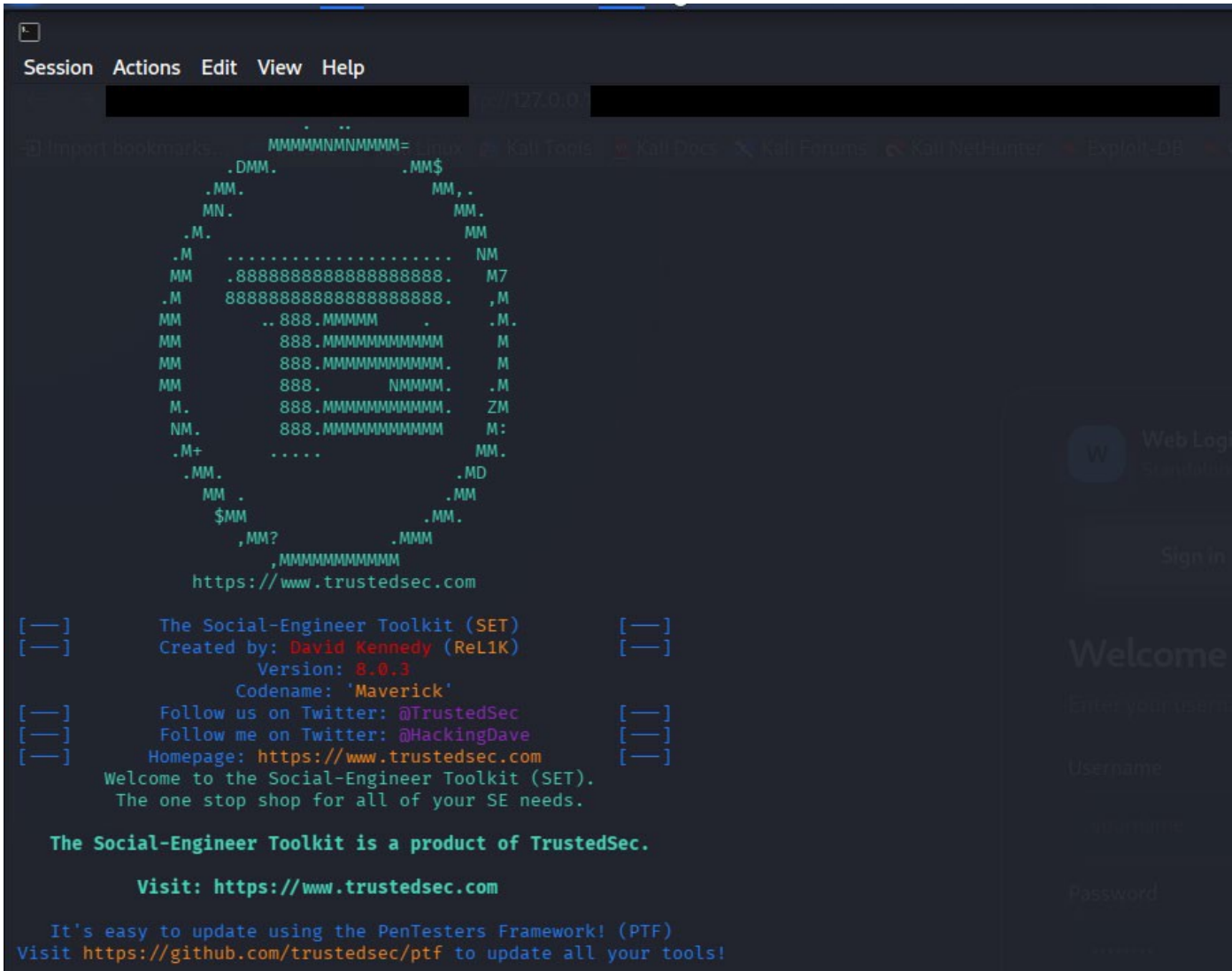


Figure 1. SET launch environment on the lab host, with local address details redacted by black bars.

```
1) Java Applet Attack Method
2) Metasploit Browser Exploit Method
3) Credential Harvester Attack Method
4) Tabnabbing Attack Method
5) Web Jacking Attack Method
6) Multi-Attack Web Method
7) HTA Attack Method

99) Return to Main Menu

set:webattack>3

The first method will allow SET to import a list of pre-defined web
applications that it can utilize within the attack.

The second method will completely clone a website of your choosing
and allow you to utilize the attack vectors within the completely
same web application you were attempting to clone.

The third method allows you to import your own website, note that you
should only have an index.html when using the import website
functionality.

1) Web Templates
2) Site Cloner
3) Custom Import

99) Return to Webattack Menu

set:webattack>2
[-] Credential harvester will allow you to utilize the clone capabilities within SET
[-] to harvest credentials or parameters from a website as well as place them into a report
```

Figure 2. SET credential-harvester attack-selection view, documenting the controlled attack path used in the lab simulation.

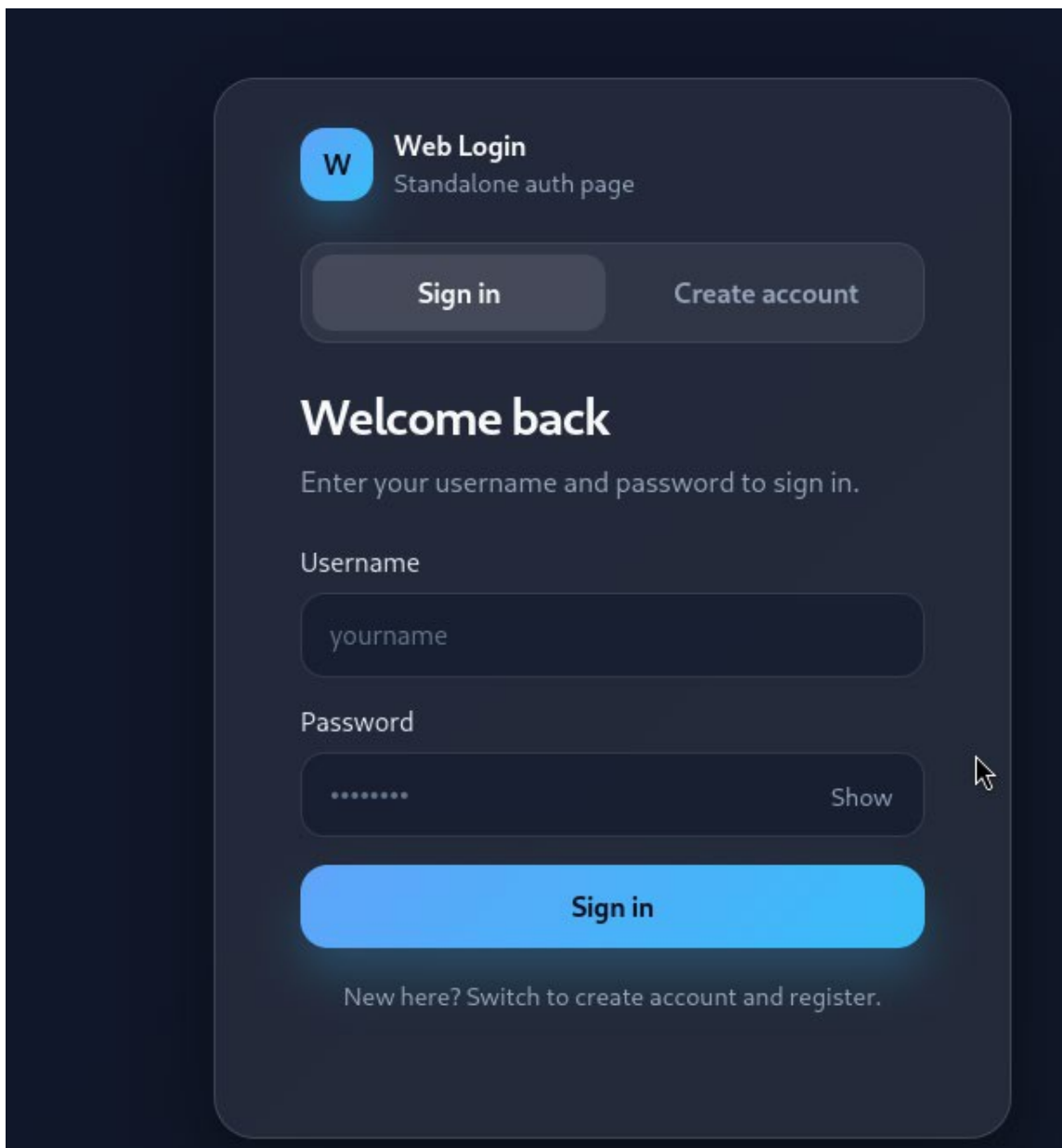


Figure 3. Source login interface used in the controlled localhost simulation.

3. Execution

As depicted in Figure 4, the cloned interface remained accessible through localhost after the credentials were submitted. Rather than redirecting the user to a destination page, the interface displayed an inline error message indicating that the server could not be reached. This is analytically important because a user could interpret the result as a routine login or connectivity failure rather than as evidence that the initial submission had already been processed.

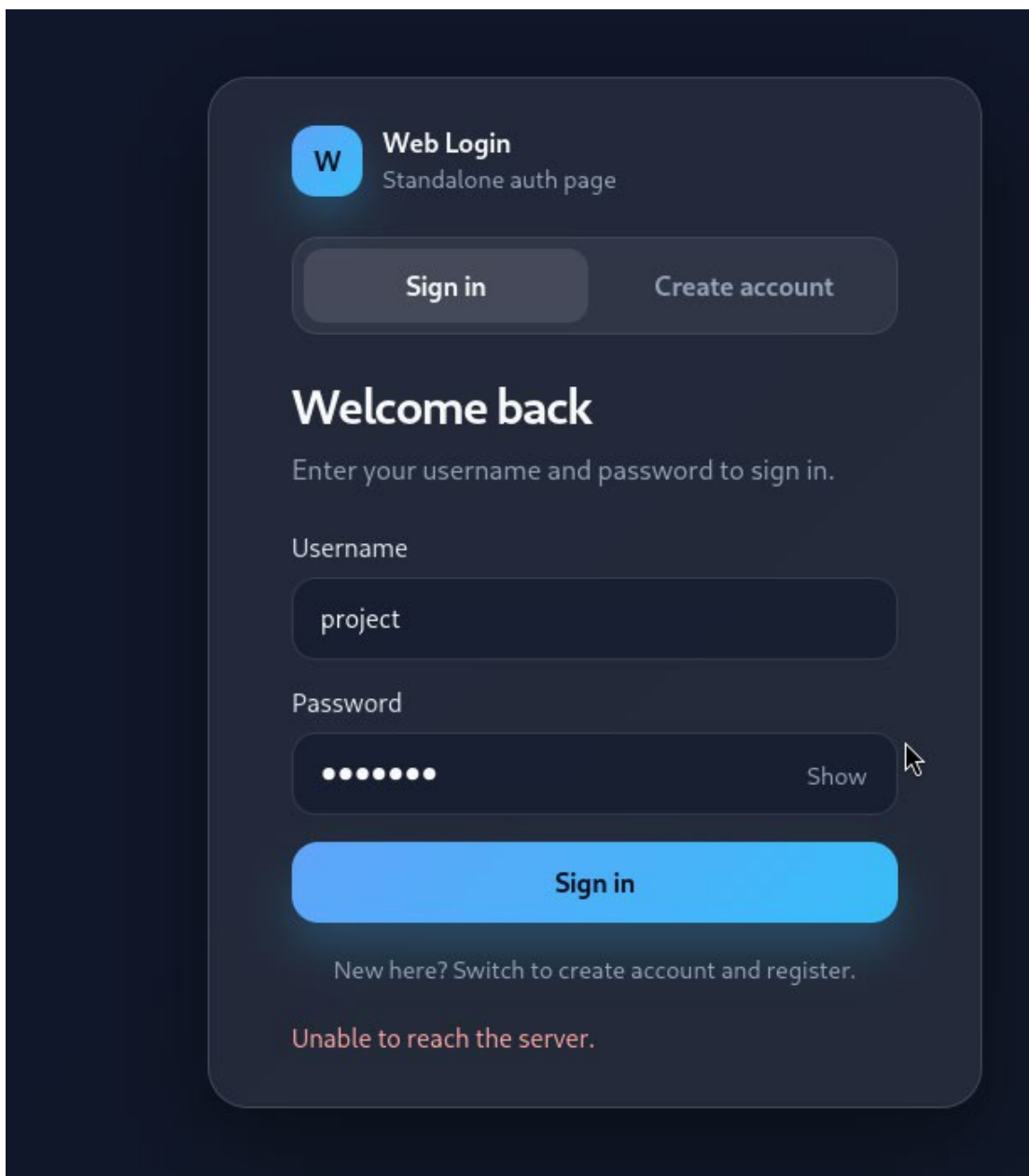


Figure 4. Localhost clone after form submission, showing an inline server-reachability error rather than a clean redirect.

4. Evidence Collected

As depicted in Figure 5, the terminal session serves as the primary forensic artifact for the case study. The output shows that the locally hosted practice page was cloned, successfully served to the browser, and followed by a recorded hit event identifying probable username and password fields from the submitted form. Local address details and submitted values were redacted before publication so the evidence remains useful without exposing unnecessary operational details.

```
— * IMPORTANT * READ THIS BEFORE ENTERING IN THE IP ADDRESS * IMPORTANT * —

The way that this works is by cloning a site and looking for form fields to
rewrite. If the POST fields are not usual methods for posting forms this
could fail. If it does, you can always save the HTML, rewrite the forms to
be standard forms and use the "IMPORT" feature. Additionally, really
important:

If you are using an EXTERNAL IP ADDRESS, you need to place the EXTERNAL
IP address below, not your NAT address. Additionally, if you don't know
basic networking concepts, and you have a private IP address, you will
need to do port forwarding to your NAT IP address from your external IP
address. A browser doesn't know how to communicate with a private IP
address, so if you don't specify an external IP address if you are using
this from an external perspective, it will not work. This isn't a SET issue
this is how networking works.

set:webattack> IP address for the POST back in Harvester/Tabnabbing [192.1
[~] SET supports both HTTP and HTTPS
[~] Example: http://www.thisisafakesite.com
set:webattack> Enter the url to clone: http://127.0.0.1:5500/public/

[*] Cloning the website: http://127.0.0.1:5500/public/
[*] This could take a little bit...

The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
127.0.0.1 - - [30/Mar/2026 21:40:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Mar/2026 21:40:04] "GET //ws HTTP/1.1" 404 -
127.0.0.1 - - [30/Mar/2026 21:40:04] "GET /favicon.ico HTTP/1.1" 404 -
[*] WE GOT A HIT! Printing the output:
POSSIBLE USERNAME FIELD FOUND: {"userna
POSSIBLE PASSWORD FIELD FOUND: {"userna
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C
```

Figure 5. Close-up terminal capture documenting the logged submission event, with local address values and submitted credential data hidden by opaque black bars.

5. Why the Exposure Occurred

The capture worked because the page being presented to the user looked like an expected sign-in interface and preserved the core interaction pattern of a normal login form. The most important weakness demonstrated here is timing: the data were exposed at submission time, before any legitimate backend storage, password hashing, or account-security controls would have become relevant.

Human factors also mattered. Users often explain away login friction as their own mistake. A page that stays visible and displays an error message can encourage another attempt instead of raising suspicion. The interface does not need to be perfect to succeed; it only needs to feel plausible for a few seconds at the moment the user is ready to type a username and password.

6. Real-World Impact to the Individual

For an individual user, credential exposure can create consequences that extend far beyond the single login form. If the captured password is reused, one submission can cascade into email takeover, password-reset abuse, fraudulent purchases, account lockout, exposure of personal documents, or compromise of cloud storage and family photos.

The impact is amplified at home because people are less formal on personal devices: they log in quickly, save passwords in browsers, share devices with relatives, follow links from text messages, and often act while distracted. The harm is not only technical. A victim may need to spend hours recovering accounts, contacting banks or service providers, explaining suspicious activity, and rebuilding trust in the devices and logins they use every day.

7. Mitigation, Patching & Prevention

The patch path for this class of exposure begins at the decision point where credentials are entered. The defensive goal is to make domain verification stronger than visual trust. Password managers that only auto-fill on the correct domain can interrupt cloned pages immediately. Passkeys, hardware security keys, and phishing-resistant multi-factor authentication reduce dependence on reusable passwords and make credential replay far harder.

Preventive controls should also address how users reach login pages. Families and small organizations should prefer bookmarks or official apps, keep unique passwords for every service, enable sign-in alerts on email and financial accounts, and treat a login error immediately after credential entry as a warning sign rather than simply a typo. At the device and network level, updated browsers and operating systems, separate browser profiles, DNS filtering, and reduced password reuse materially lower the risk of a single convincing page becoming a larger account compromise.

8. Ethical Consideration

This case study was conducted in a controlled environment strictly for educational and analytical purposes. Both the source and cloned login interfaces were hosted locally, and all credentials used during testing were self-generated. No real users, external systems, public hosts, or third-party services were involved. The screenshots and terminal outputs presented in this report are treated as research artifacts from a bounded simulation rather than instructions for real-world application.

The findings confirm that a cloned login interface can capture credentials at the point of entry, even when the post-submission behavior is imperfect or produces visible errors. The significance of this case is forensic and defensive: it illustrates how believable sign-in flows can expose sensitive information before trusted backend protections become relevant. The defensive lesson is straightforward: verify the destination, reduce password reuse, and treat unexpected login errors as possible detection clues rather than harmless background noise.